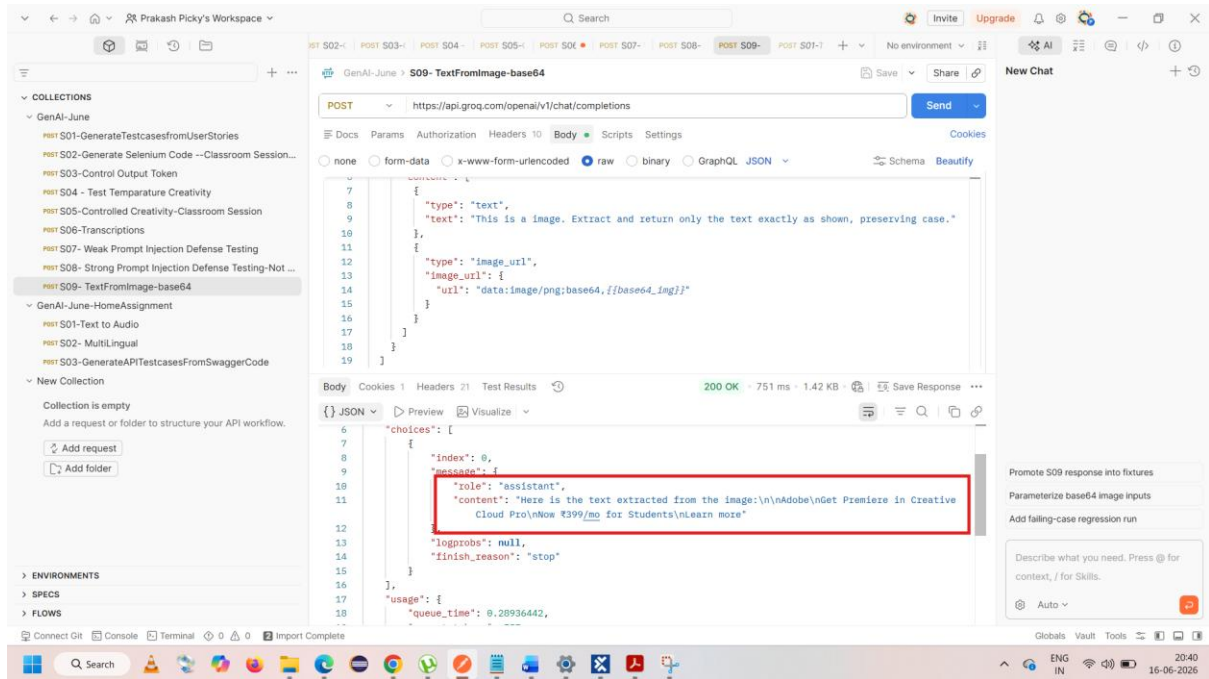




Adobe

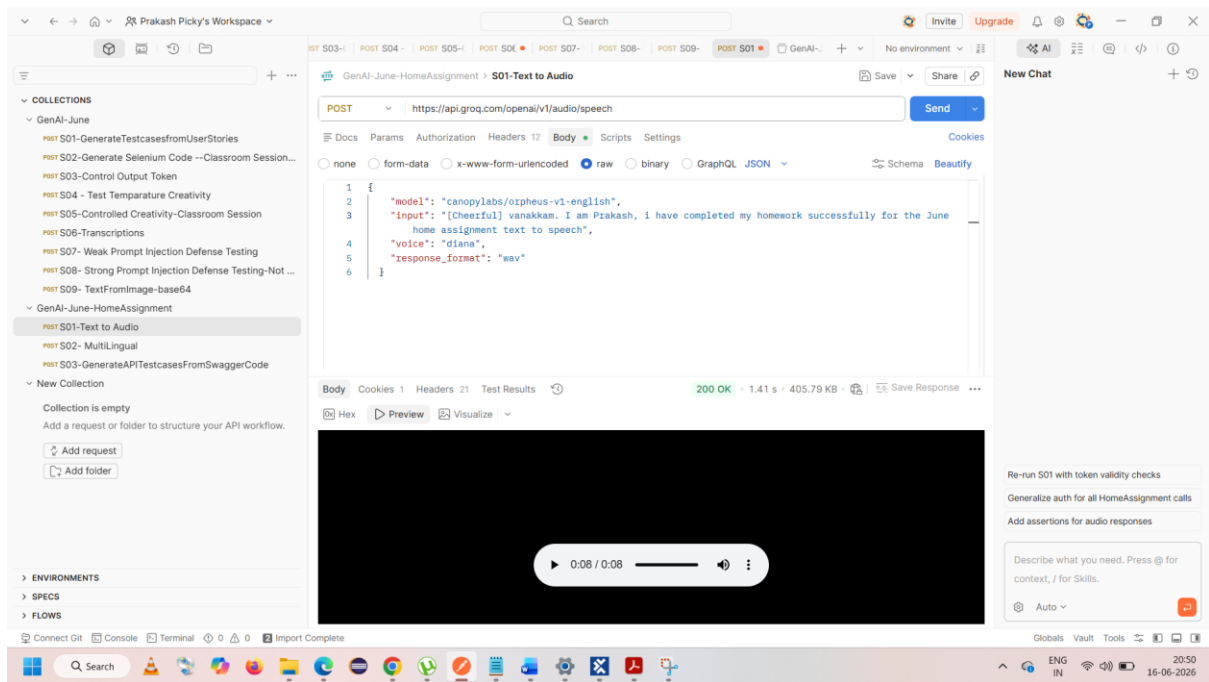
Get Premiere in Creative Cloud Pro

Now ₹399/mo for Students

[Learn more](#)

=====

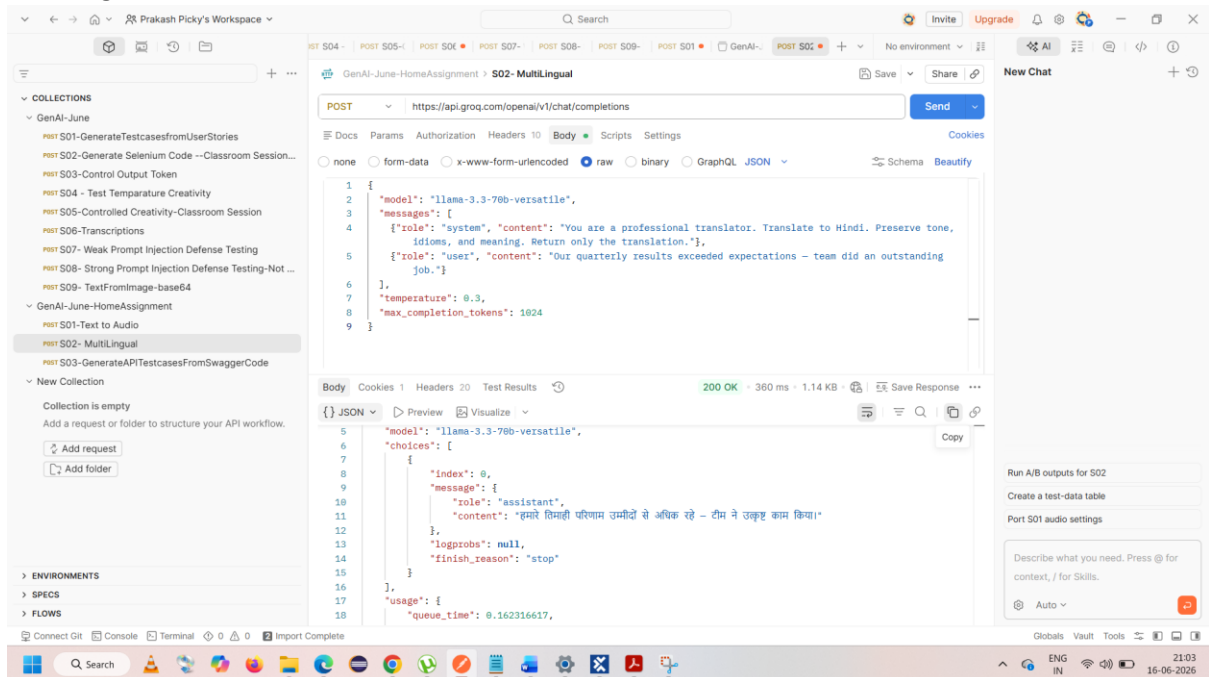
Text to voice



response.mp3

Working successfully ===

Multilingual

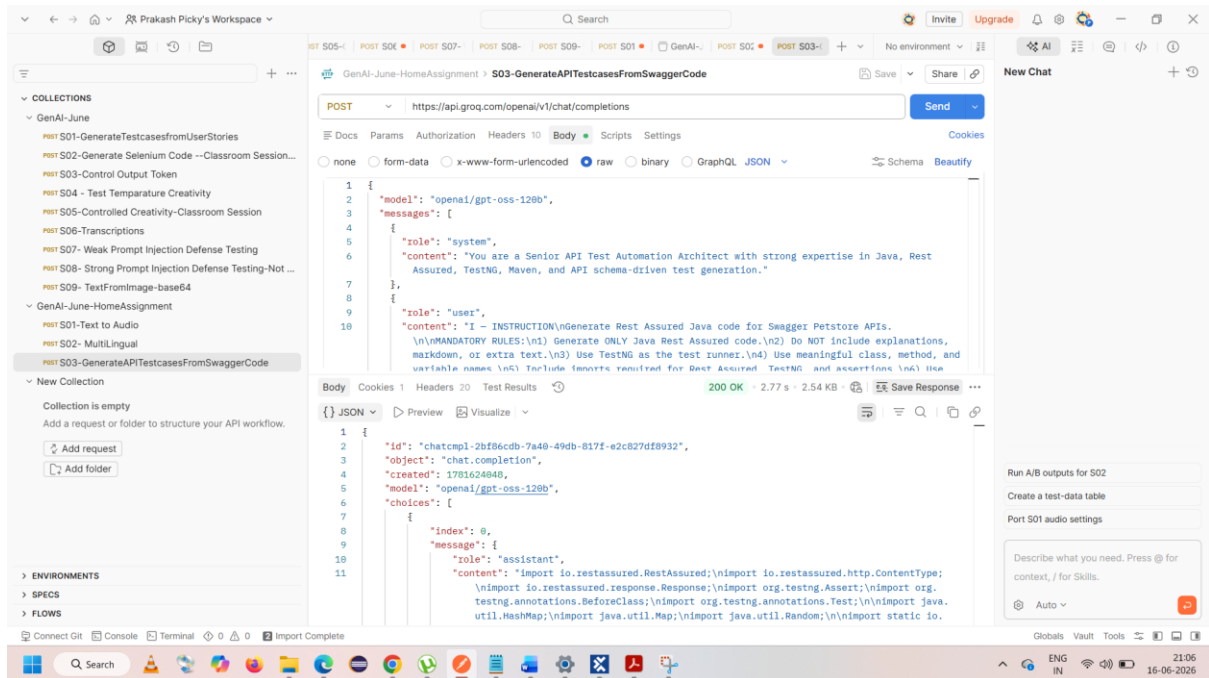


Result:

```
{  
  "id": "chatcmpl-1257901c-ec05-4353-b5c6-17aeffd22d4c",  
  "object": "chat.completion",  
  "created": 1718880000,  
  "model": "gpt-4o-mini",  
  "usage": {  
    "prompt_tokens": 10,  
    "completion_tokens": 10,  
    "total_tokens": 20  
  },  
  "choices": [ {  
    "index": 0,  
    "message": {  
      "role": "assistant",  
      "content": "हमारे तिमाही परिणाम उम्मीदों से अधिक रहे - टीम ने उत्कृष्ट काम किया।"  
    },  
    "logprobs": null,  
    "finish_reason": "stop"  
  } ]  
}
```

```
"object": "chat.completion",
"created": 1781623992,
"model": "llama-3.3-70b-versatile",
"choices": [
  {
    "index": 0,
    "message": {
      "role": "assistant",
      "content": "हमारे तिमाही परिणाम उम्मीदों से अधिक रहे — टीम ने उत्कृष्ट काम किया।"
    },
    "logprobs": null,
    "finish_reason": "stop"
  }
],
"usage": {
  "queue_time": 0.162316617,
  "prompt_tokens": 71,
  "prompt_time": 0.00365889,
  "completion_tokens": 35,
  "completion_time": 0.081754473,
  "total_tokens": 106,
  "total_time": 0.085413363
},
"usage_breakdown": null,
"system_fingerprint": "fp_3272ea2d91",
"x_groq": {
  "id": "req_01kv8h1bebf8hr8kepr5jrj8x4",
  "seed": 2068435456
},
"service_tier": "on_demand"
}
```

Swagger api code



Result:

```
{
  "id": "chatcmpl-2bf86cdb-7a40-49db-817f-e2c827df8932",
  "object": "chat.completion",
  "created": 1781624048,
  "model": "openai/gpt-oss-120b",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "import io.restassured.RestAssured;\nimport io.restassured.http.ContentType;\nimport io.restassured.response.Response;\nimport org.testng.Assert;\nimport org.testng.annotations.BeforeClass;\nimport org.testng.annotations.Test;\n\nimport java.util.HashMap;\nimport java.util.Map;\nimport java.util.Random;\n\npublic class PetApiCrudTest {\n\n    private static final String BASE_URI = \"https://petstore.swagger.io\";\n    private static long petId;\n\n    @BeforeClass\n    public void setup() {\n        RestAssured.baseURI = BASE_URI;\n        petId = new Random().nextLong() & Long.MAX_VALUE; // ensure positive\n    }\n\n    private"
      }
    }
  ]
}
```

```

Map<String, Object> buildPetPayload(String name, String status) {
    Map<String, Object> category = new HashMap<>();
    category.put("id", 0);
    category.put("name", "string");
    Map<String, Object> tag = new HashMap<>();
    tag.put("id", 0);
    tag.put("name", "string");
    Map<String, Object> pet = new HashMap<>();
    pet.put("id", petId);
    pet.put("category", category);
    pet.put("name", name);
    pet.put("photoUrls", new String[]{"string"});
    pet.put("tags", new Object[]{tag});
    pet.put("status", status);
    return pet;
}

@Test(priority = 1)
public void createPet() {
    Map<String, Object> requestBody = buildPetPayload("Fluffy", "available");
    Response response = given().contentType(ContentType.JSON).body(requestBody).when().post("/v2/pet").then().extract().response();
    Assert.assertEquals(response.getStatusCode(), 200, "Create pet status code");
    Assert.assertEquals(response.jsonPath().getLong("id"), petId, "Pet ID mismatch");
    Assert.assertEquals(response.jsonPath().getString("name"), "Fluffy", "Pet name mismatch");
    Assert.assertEquals(response.jsonPath().getString("status"), "available", "Pet status mismatch");
}

@Test(priority = 2, dependsOnMethods = {"createPet"})
public void getPetById() {
    Response response = given().accept(ContentType.JSON).when().get("/v2/pet/{petId}", petId).then().extract().response();
    Assert.assertEquals(response.getStatusCode(), 200, "Get pet status code");
    Assert.assertEquals(response.jsonPath().getLong("id"), petId, "Pet ID mismatch");
    Assert.assertEquals(response.jsonPath().getString("name"), "Fluffy", "Pet name mismatch");
    Assert.assertEquals(response.jsonPath().getString("status"), "available", "Pet status mismatch");
}

@Test(priority = 3, dependsOnMethods = {"getPetById"})
public void updatePet() {
    Map<String, Object> requestBody = buildPetPayload("FluffyUpdated", "sold");
    Response response = given().contentType(ContentType.JSON).body(requestBody).when().put("/v2/pet/{petId}", petId).then().extract().response();
    Assert.assertEquals(response.getStatusCode(), 200, "Update pet status code");
    Assert.assertEquals(response.jsonPath().getLong("id"), petId, "Pet ID mismatch after update");
    Assert.assertEquals(response.jsonPath().getString("name"), "FluffyUpdated", "Pet name not updated");
    Assert.assertEquals(response.jsonPath().getString("status"), "sold", "Pet status not updated");
}

@Test(priority = 4, dependsOnMethods = {"updatePet"})
public void deletePet() {
    Response response = given().when().delete("/v2/pet/{petId}", petId).then().extract().response();
    Assert.assertEquals(response.getStatusCode(), 200, "Delete pet status code");
    Assert.assertEquals(response.jsonPath().getString("message"), String.valueOf(petId), "Delete response message mismatch");
}

```

"reasoning": "We need to output Java code only, no markdown fences. Provide imports, class with @BeforeClass to set baseUrl, and test methods with @Test, using Rest Assured, TestNG, and assertions. Use meaningful names. We'll generate code that uses static imports for RestAssured.given, etc., and use Assert from TestNG. Also need to create request bodies (JSON) for pet. Use a petId variable shared across tests. Use @Test(dependsOnMethods) to ensure order. Provide CRUD. Use content type JSON. Use proper path: /v2/pet. Delete path: /v2/pet/{petId}. For update, same as create but using POST? In Swagger Petstore, update uses PUT /pet. We'll use PUT. We'll create a POJO for pet? Could just use Map or JSON string. Simpler: use a Map and RestAssured will serialize. Use a method to build pet payload. Make class: PetApiCrudTest. Add private static final String BASE_URI = "https://petstore.swagger.io"; private static long petId; generate random id. Implement

createPet: POST /v2/pet with body, assert status 200, response contains id and name.\n\nGet pet: GET /v2/pet/{petId}, assert status 200 and name.\n\nUpdate pet: PUT /v2/pet with changed name/status, assert status 200.\n\nDelete pet: DELETE /v2/pet/{petId}, assert status 200.\n\nInclude imports.\n\nLet's craft code.\n\n"

```
    },
    "logprobs": null,
    "finish_reason": "stop"
  }
],
"usage": {
  "queue_time": 0.158977649,
  "prompt_tokens": 396,
  "prompt_time": 0.017776857,
  "completion_tokens": 1216,
  "completion_time": 2.548851071,
  "total_tokens": 1612,
  "total_time": 2.566627928,
  "completion_tokens_details": {
    "reasoning_tokens": 318
  }
},
"usage_breakdown": null,
"system_fingerprint": "fp_4727af4560",
"x_groq": {
  "id": "req_01kv8h30g5esstpjtwyh4kaj2y",
  "seed": 126252048
},
"service_tier": "on_demand"
}
```
